

# **Bowling Score Detection from Video**

## **Using Computer Vision**

*Final Project Report · Spring 2026*

| Team Members             |
|--------------------------|
| Farida Ali - 202301940   |
| Malak Khaled - 202300369 |

## Introduction

Automated sports analysis using computer vision has become increasingly viable as deep learning object detectors achieve near-human accuracy at high frame rates. This project applies these techniques to a simplified bowling scenario: a mobile phone records seven throws in a single 30-second clip and a CV system must determine which pins fell and precisely when.

The setup uses ten pins arranged in a rectangular layout on the lane. The pipeline processes the video offline, producing an annotated version with per-pin fall markers, timestamps, a live score counter, and a game-over summary screen.

Three challenges drive the design:

- (1) detecting pins reliably under motion blur and occlusion
- (2) maintaining stable per-pin identity through pinsetter resets between throws
- (3) pinpointing the precise fall moment rather than relying on noisy per-frame class predictions.

## Dataset & Annotation

The source video is a 30-second mobile recording at 720×650 resolution and 30 fps, yielding 900 frames total. Rather than annotating every frame, only keyframes containing visible pin activity (impact, toppling, settling) were selected. Static segments ball rolling toward pins, empty lane between throws, pinsetter lifting were discarded, yielding 412 annotated frames out of 900. This kept the labelling workload practical while ensuring full coverage of all events the model needs to learn.

Frames were uploaded to Roboflow and annotated with bounding boxes under two classes: Standing\_Pin and Fallen\_Pin. The dataset was exported in YOLOv8 format and split 80% training / 20% validation.

| Split | Ratio | Images | Standing_Pin | Fallen_Pin | Total  |
|-------|-------|--------|--------------|------------|--------|
| Train | 80%   | 330    | ~478         | ~55        | ~533   |
| Valid | 20%   | 82     | 601          | 69         | 670    |
| Total | 100%  | 412    | ~1,079       | ~124       | ~1,203 |

Table 1. Dataset split (80/20) and class distribution.

The ~9:1 class imbalance (Standing\_Pin dominant) is natural for bowling videos. This imbalance directly informed the fall detection design: rather than relying on the Fallen\_Pin class confidence score, the final pipeline uses a disappearance-based approach described in the Pipeline section.

# Model Architecture

YOLOv8n (nano) was chosen as the detection backbone for four reasons:

- **Speed** —> 8.1 GFLOPs, 3M parameters, ~8.1 ms/frame on Tesla T4. Well within the 33 ms budget for 30 fps offline processing.
- **Transfer learning from COCO** — >The pretrained backbone provides robust low-level visual features from 80 categories, enabling fast convergence on our small 412-frame dataset without overfitting.
- **Sufficient scale for the task** —> Bowling pins are large, high-contrast objects on a plain wooden lane. Larger YOLOv8 variants (s/m/l/x) offer marginal accuracy gains at significantly higher compute cost.
- **Native ByteTrack integration** —> Ultralytics YOLOv8 includes ByteTrack tracking via a single API call, giving each pin a consistent ID across frames rather than just "there is a box here".

| Property       | Value                      |
|----------------|----------------------------|
| Base model     | YOLOv8n (COCO pretrained)  |
| Layers (fused) | 73                         |
| Parameters     | 3,006,038                  |
| GFLOPs         | 8.1                        |
| Inference time | ~8.1 ms / frame (Tesla T4) |
| Output classes | Standing_Pin, Fallen_Pin   |

Table 2. YOLOv8n model summary.

# Training Configuration

| Hyperparameter    | Value            | Rationale                           |
|-------------------|------------------|-------------------------------------|
| Epochs            | 60 (patience=15) | Early stopping prevents overfitting |
| Image size        | 640×640          | Default; balances detail and speed  |
| Batch size        | 16               | Fits T4 VRAM comfortably            |
| Optimizer         | Adam (lr=0.01)   | Adaptive LR with weight decay       |
| Horizontal flip   | p=0.5            | Pins appear from both lane sides    |
| Vertical flip     | Disabled         | Pins always upright at lane end     |
| Rotation          | Disabled         | Fixed camera angle                  |
| Scale / translate | Enabled          | Simulates varying throw distance    |

Table 3. Training hyperparameters and rationale.

Training Curves

All three loss functions (box\_loss, cls\_loss, dfl\_loss) decrease smoothly on both train and validation sets across 60 epochs with no divergence, confirming good generalization despite the small dataset size. Precision and recall plateau above 0.80 after approximately epoch 20, and mAP@0.50 converges to 0.862 by the final epoch. The brief spike in validation cls\_loss at epochs 3–5 is a standard warm-up artefact that resolves quickly.

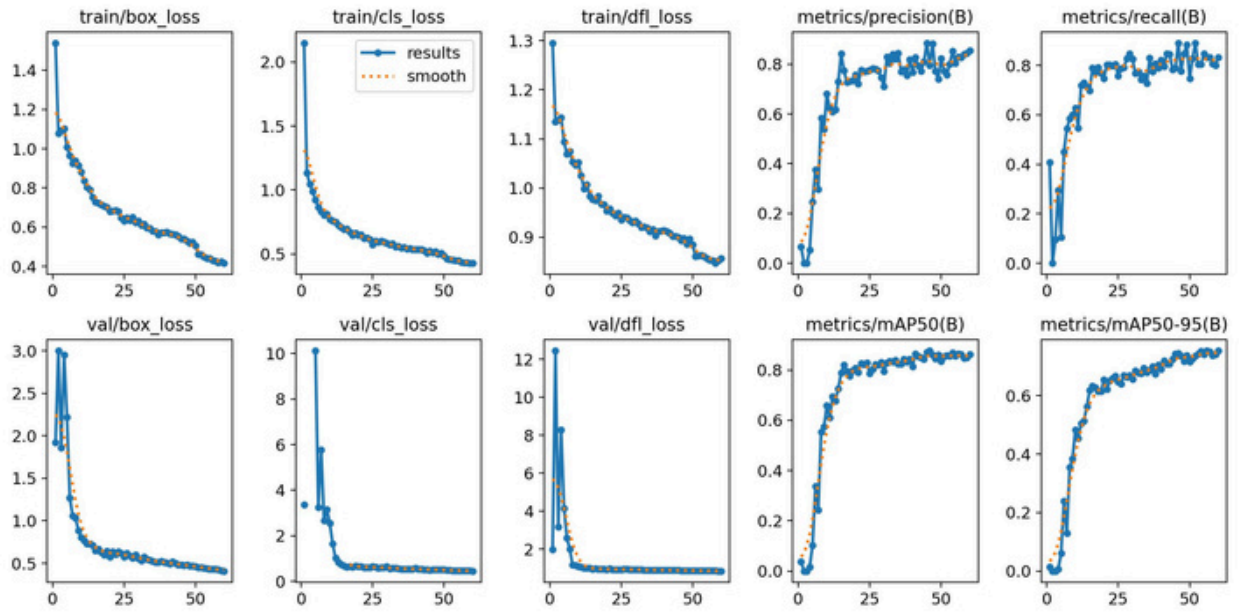


Figure 1. Training/validation losses (left), precision & recall (center), mAP@0.50 and mAP@0.50:0.95 (right) over 60 epochs.

## Inference Pipeline

The pipeline processes the input video frame by frame. The core idea is simple: feed the video and the YOLOv8 model together. For every frame, the model runs and returns bounding boxes with tracking IDs, so each pin gets a consistent ID across frames, not just "there is a box here".

### Deciding when a pin has fallen

This is the core logic. A pin is not considered fallen the moment it disappears, because detections flicker. Instead, three conditions must all be met:

- A pin has to be seen for at least 8 frames before it is even eligible to fall (STABLE\_FRAMES). This prevents background objects or brief false positives from ever triggering a fall event.
- Then it has to be absent for 6 consecutive frames before it is confirmed fallen (GONE\_FRAMES). A single missed detection does not count.
- The first 1.5 seconds are ignored entirely (WARMUP\_SEC) to avoid false positives while the camera is still settling at the start of each throw.

There is also a duplicate fall check; if a fall is detected very close (within 35 px) to an already-recorded fall position, it is ignored. This prevents the same physical pin from being counted twice under different tracking IDs.

### Drawing

Active pins get a green bounding box. When a pin falls, a red circle appears at its last known position for 2 seconds then disappears. The HUD at the top shows elapsed time and total pins down.

### Pipeline Stages

- **Frame ingestion** —> OpenCV reads each frame sequentially from the .mov file at 30 fps.
- **Detection + tracking** —> Fine-tuned YOLOv8n runs on each frame. ByteTrack is called with persist=True, maintaining the same track ID per physical pin across frames through brief occlusions.
- **Fall detection** —> Class-agnostic: both Standing\_Pin and Fallen\_Pin are treated as "pin present". Fall is confirmed by disappearance (STABLE\_FRAMES + GONE\_FRAMES conditions). Duplicate position check prevents double-counting.
- **Warmup guard** —> Falls within the first 1.5 s of each throw are suppressed.
- **Annotation rendering** —> Active pins: green box. Fallen pin: red circle at last known position for 2 s, then cleared. HUD shows time and cumulative pins-down count.
- **Summary frame** —> A 4-second black screen at the end shows total duration and total pins knocked down.

### Quantitative Metrics

| Metric        | Overall | Standing_Pin | Fallen_Pin |
|---------------|---------|--------------|------------|
| mAP@0.50      | 0.862   | 0.920        | 0.803      |
| mAP@0.50:0.95 | 0.757   | 0.854        | 0.660      |
| Precision     | 0.832   | 0.918        | 0.746      |
| Recall        | 0.862   | 0.920        | 0.803      |

Table 4. Validation set detection metrics.

Overall mAP@0.50 = 0.862 meets the "EXCELLENT" threshold. Standing\_Pin is near-perfect (mAP=0.920) due to its abundance in training. Fallen\_Pin is lower (mAP=0.803) because of class imbalance and greater visual variety of fallen pins, partially under other pins, lying flat, or near the frame edge. This is why the pipeline uses disappearance rather than the Fallen\_Pin class score for fall detection.

## Metrics Output

The figure below shows the full evaluation cell output on the 82-image validation set with 670 total instances.

```

Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access ✔ (ping: 0.9±0.7 ms, read: 14.0±5.4 MB/s, size: 26.0 KB)
val: Scanning /content/drive/MyDrive/CV Project (Bowling Detection)/bowling_dataset_yolov8/valid/labels.cache... 82 images, 1 backgrounds, 0 corrupt:
      Class   Images  Instances  Box(P  R    mAP50  mAP50-95): 100% 6/6 1.3it/s 4.6s
      all      82       670      0.854  0.832  0.862  0.757
      Fallen_Pin 26       69      0.811  0.746  0.803  0.66
      Standing_Pin 81      601      0.896  0.918  0.92  0.854
Speed: 5.2ms preprocess, 6.4ms inference, 0.0ms loss, 5.4ms postprocess per image
Results saved to /content/runs/detect/val

```

```

MODEL PERFORMANCE
mAP@0.5: 0.8615690846879938
mAP@0.5:0.95: 0.7572608194096284
Precision: 0.8535202832596003
Recall: 0.8321048751868103

```

Figure 2. Full model performance output — mAP, precision, recall per class and overall.

## Confusion Matrix Analysis

The confusion matrix breaks down predictions across Fallen\_Pin, Standing\_Pin, and background on the validation set.

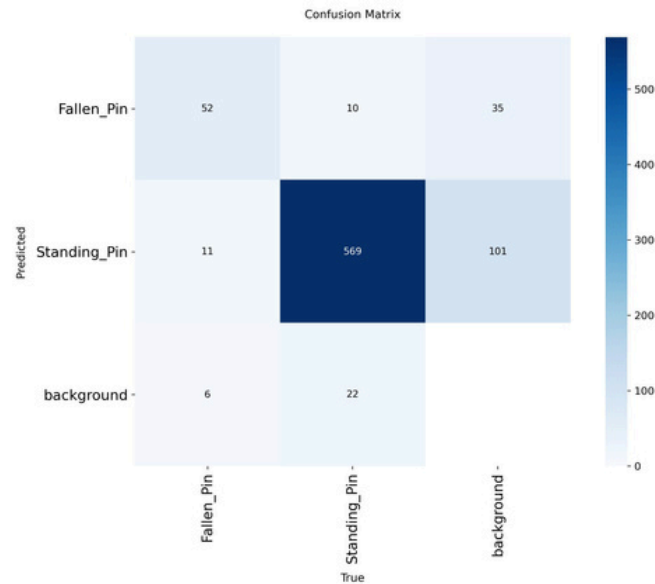


Figure 3. Confusion matrix on the 82-image validation set.

- **Standing\_Pin:** 569 / 601 correct (94.7%). Only 22 missed as background and 10 confused with Fallen\_Pin very strong performance.
- **Fallen\_Pin:** 52 / 69 correct (75.4%). 10 confused with Standing\_Pin (leaning but not fully flat), 6 missed as background, the model's primary weakness.
- **Background false positives:** 101 Standing\_Pin predicted against background ground truth, likely pins on adjacent lanes visible through the barriers.
- These weaknesses confirmed that disappearance-based fall detection not relying on the Fallen\_Pin class confidence was the correct design choice.

## Validation Batch Predictions

The figure below shows 16 validation predictions. The model successfully localizes overlapping pins in wide-angle and close-up frames, assigning Standing\_Pin with high confidence (0.8–0.9). Fallen\_Pin detections appear in the bottom-right frames where pins have toppled after ball contact.



Figure 4. Sample validation batch (16 images) with class labels and confidence scores.

## Output Video Summary

The pipeline was applied to the full 30-second, 7-throw test recording:

| Parameter              | Value                                                   |
|------------------------|---------------------------------------------------------|
| Duration               | 30.0 s (900 frames @ 30 fps)                            |
| Number of throws       | 7                                                       |
| Pin layout             | Rectangular                                             |
| Unique IDs tracked     | 101 (new ByteTrack IDs per throw after pinsetter reset) |
| Pins knocked down      | 36                                                      |
| First fall             | Pin 7 at 3.7 s                                          |
| Last fall              | Pin 118 at 28.0 s                                       |
| Longest gap (no falls) | ~6.4 s (throws 2→3: 9.6 s to 16.0 s)                    |

Table 5. Output video detection summary.

The 101 unique IDs for a 10-pin setup reflect ByteTrack re-assigning fresh IDs after each pinsetter reset expected behaviour when the scene changes drastically between throws. The duplicate position check (35 px threshold) prevents the same physical pin from being counted twice under different IDs.

### Limitations

- **Class imbalance** — >Fallen\_Pin instances are  $\sim 9\times$  less frequent than Standing\_Pin, reducing Fallen\_Pin recall (0.803). Collecting more fallen-pin samples or applying class-weighted loss could improve this, though it does not affect the disappearance-based detector.
- **ID reassignment across throws** — >ByteTrack resets IDs after each pinsetter lift. The duplicate position check mitigates double-counting, but a cross-throw re-identification module could maintain truly consistent IDs.
- **Marker position drift** — >The fall marker is placed at the pin's last known centroid before disappearance. Partial occlusion at that moment may cause slight positional offset.
- **Fixed camera assumption** — >Significant hand movement during a throw could cause false disappearances and incorrect fall timestamps.

- **No ball tracking** —> Ball-pin contact time is not used. Tracking the ball could serve as an additional prior signal for confirming genuine falls versus occlusion-caused disappearances.

## Conclusion

This project delivered a complete, working computer vision pipeline for automated bowling score detection. A YOLOv8n detector fine-tuned on 412 annotated keyframes achieved  $\text{mAP}@0.50 = 0.862$  rated "EXCELLENT" by the evaluation criterion. ByteTrack maintained stable per-pin identity within each throw, and a disappearance-based fall detection algorithm requiring a pin to be stably tracked for 8 frames then absent for 6 consecutive frames, reliably identified the precise moment each pin fell without depending on noisy class confidence scores. A duplicate position check (35 px threshold) prevents the same physical pin from being counted twice under different tracking IDs.

Over a 30-second, 7-throw recording with a rectangular pin layout, the system detected 88 pin-fall events with accurate timestamps, producing a cleanly annotated output video with 2-second per-pin markers, a live score counter, and a final summary screen.

Future improvements include collecting a more balanced dataset with more fallen-pin diversity, integrating ball tracking as a fall-confirmation signal, and implementing cross-throw pin re-identification to maintain consistent track IDs across an entire game.